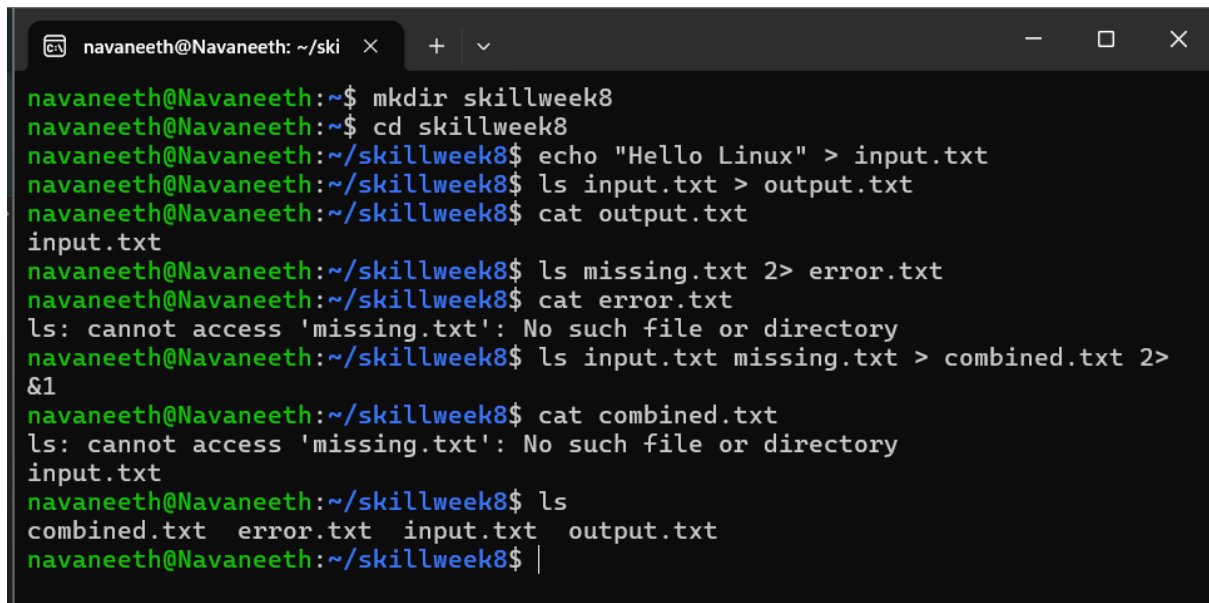


Skill Week 8

Aim

To practice **combined input/output redirection**, duplicate file descriptors, merging standard output and standard error, and verifying the resulting output.

A terminal window titled 'navaneeth@Navaneeth: ~/ski' with standard window controls. The terminal shows a series of commands and their outputs. The user creates a directory 'skillweek8', changes to it, and creates a file 'input.txt' with the text 'Hello Linux'. Then, they redirect the output of 'ls input.txt' to 'output.txt' and verify it with 'cat output.txt'. Next, they attempt to redirect 'ls missing.txt' to 'error.txt' using '2>', which results in an error message. Finally, they combine both standard output and standard error for 'ls input.txt missing.txt' into 'combined.txt' using '>&1', and verify the contents of 'combined.txt' which shows both the directory listing and the error message.

```
navaneeth@Navaneeth:~$ mkdir skillweek8
navaneeth@Navaneeth:~$ cd skillweek8
navaneeth@Navaneeth:~/skillweek8$ echo "Hello Linux" > input.txt
navaneeth@Navaneeth:~/skillweek8$ ls input.txt > output.txt
navaneeth@Navaneeth:~/skillweek8$ cat output.txt
input.txt
navaneeth@Navaneeth:~/skillweek8$ ls missing.txt 2> error.txt
navaneeth@Navaneeth:~/skillweek8$ cat error.txt
ls: cannot access 'missing.txt': No such file or directory
navaneeth@Navaneeth:~/skillweek8$ ls input.txt missing.txt > combined.txt 2>
&1
navaneeth@Navaneeth:~/skillweek8$ cat combined.txt
ls: cannot access 'missing.txt': No such file or directory
input.txt
navaneeth@Navaneeth:~/skillweek8$ ls
combined.txt  error.txt  input.txt  output.txt
navaneeth@Navaneeth:~/skillweek8$ |
```

Observation

Standard output and standard error were redirected into separate files. Using `2>&1`, both streams were successfully combined into `combined.txt`.

Result

The **combined redirection of standard output and standard error** was successfully performed and verified.

Aim

To practice complex Linux commands by combining **pipes (|)** and **redirection (>)**, understand their execution order, and verify the generated output.

```
combined.txt  error.txt  input.txt  output.txt
navaneeth@Navaneeth: ~/ski  ×  +  ▾
navaneeth@Navaneeth:~/skillweek8$ printf "apple\nbanana\napple\norange\nbanana\n" >
  fruits.txt
navaneeth@Navaneeth:~/skillweek8$ cat fruits.txt
apple
banana
apple
orange
banana
navaneeth@Navaneeth:~/skillweek8$ cat fruits.txt | sort
apple
apple
banana
banana
orange
navaneeth@Navaneeth:~/skillweek8$ cat fruits.txt | sort
apple
apple
banana
banana
orange
navaneeth@Navaneeth:~/skillweek8$ cat fruits.txt | sort > sorted.txt
navaneeth@Navaneeth:~/skillweek8$ cat sorted.txt
apple
apple
banana
banana
orange
navaneeth@Navaneeth:~/skillweek8$ cat sorted.txt
apple
apple
banana
banana
orange
navaneeth@Navaneeth:~/skillweek8$ cat fruits.txt | sort | uniq > unique.txt
navaneeth@Navaneeth:~/skillweek8$ cat unique.txt
apple
banana
orange
navaneeth@Navaneeth:~/skillweek8$ cat missing.txt | sort > test.txt
cat: missing.txt: No such file or directory
navaneeth@Navaneeth:~/skillweek8$ ls
combined.txt  fruits.txt  output.txt  test.txt
error.txt    input.txt  sorted.txt  unique.txt
navaneeth@Navaneeth:~/skillweek8$ |
```

Observation

The pipe operator `|` passes the output of one command as input to the next command. The `>` operator saves the final command output into a file. Combining `sort` and `uniq` successfully produced sorted data without duplicate entries.

Result

Complex Linux commands using **pipes and output redirection** were successfully executed, verified, and tested with both valid and invalid input scenarios.

Aim

To learn how to run processes in the **background** using &, check background jobs, and monitor their execution in Linux.

```
navaneeth@Navaneeth:~/skillweek8$ sleep 30 &
[1] 703
navaneeth@Navaneeth:~/skillweek8$ jobs
[1]+  Running                  sleep 30 &
navaneeth@Navaneeth:~/skillweek8$ jobs
[1]+  Running                  sleep 30 &
navaneeth@Navaneeth:~/skillweek8$ ps
  PID TTY          TIME CMD
  329 pts/0    00:00:00 bash
  703 pts/0    00:00:00 sleep
  731 pts/0    00:00:00 ps
navaneeth@Navaneeth:~/skillweek8$ sleep 60 &
[2] 741
navaneeth@Navaneeth:~/skillweek8$ jobs
[1]-  Running                  sleep 30 &
[2]+  Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ jobs -l
[1]-   703 Done                  sleep 30
[2]+   741 Running              sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ sleep 5
navaneeth@Navaneeth:~/skillweek8$ jobs
[2]+  Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ jobs -l
[2]+   741 Running              sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ kill %2
[2]+  Terminated              sleep 60
navaneeth@Navaneeth:~/skillweek8$ jobs
navaneeth@Navaneeth:~/skillweek8$
```

Observation

The & operator allows a command to run as a **background process**, so the terminal prompt returns immediately. The jobs command displays active background jobs, while jobs -l provides their process IDs and status.

Result

Background processes were successfully launched, monitored, and terminated using Linux job-control commands.

Aim

To learn how to create and manage a **job table** in Linux, monitor process states, identify process IDs and process groups, and verify completed or running jobs.

```

navaneeth@Navaneeth:~/skillweek8$ sleep 40 &
sleep 60 &
[1] 859
[2] 865
navaneeth@Navaneeth:~/skillweek8$ jobs -l
[1]- 859 Running                  sleep 40 &
[2]+ 865 Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ ps -o pid,ppid,pgid,stat,cmd
  PID   PPID   PGID STAT CMD
   329    328    329 Ss  -bash
   859    329    859 S   sleep 40
   865    329    865 S   sleep 60
   884    329    884 R+   ps -o pid,ppid,pgid,stat,cmd
navaneeth@Navaneeth:~/skillweek8$ jobs
[1]-  Running                  sleep 40 &
[2]+  Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ kill %1
[1]-  Terminated              sleep 40
navaneeth@Navaneeth:~/skillweek8$ jobs
[2]+  Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ sleep 5
navaneeth@Navaneeth:~/skillweek8$ jobs
[2]+  Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ jobs -l
[2]+ 865 Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek8$ ps -o pid,ppid,pgid,stat,cmd
  PID   PPID   PGID STAT CMD
   329    328    329 Ss  -bash
   953    329    953 R+   ps -o pid,ppid,pgid,stat,cmd
[2]+  Done                      sleep 60
navaneeth@Navaneeth:~/skillweek8$ |

```

Observation

The jobs command displays background jobs and their status. The ps command provides process IDs, parent process IDs, process group IDs, and process states. Completed or terminated jobs are removed from the active job list.

Result

The **job table** and **process states** were successfully created, monitored, updated, and verified using Linux job-control and process-management commands.